

PRODUCT BRIEF · 2026

# MiniMax Code

## AGENTIC CODING WORKSPACE

A multi-model agentic workspace where engineering teams ship production software with AI teammates — autonomous loops, durable memory, and a tool-rich harness around the model.

**1-M**

UNIFIED MODEL  
LAYER

**6+**

CAPABILITY  
SURFACES

**∞**

SESSION  
MEMORY

VERSION brief-2026.07

AUTHOR Mavis · mavis

RELEASE Public · v1  
ready

# Inside this report

A six-section brief on what MiniMax Code is, how it's built, and where it's going. Every section is a self-contained read — skip around using the links below.

|    |                                         |                                                                                               |      |
|----|-----------------------------------------|-----------------------------------------------------------------------------------------------|------|
| 01 | <b>Mission &amp; Positioning</b>        | What problem MiniMax Code solves, who it's for, and the three beliefs it ships with.          | P.04 |
| 02 | <b>Core Capabilities</b>                | Agentic loop, multi-agent orchestration, persistent memory, and skills extensibility.         | P.05 |
| 03 | <b>Architecture &amp; Stack</b>         | Five layers from foundation model up to surface — and why each layer is its own product.      | P.06 |
| 04 | <b>Developer Workflows</b>              | Autonomy levels, recommended entry paths, and how teams actually use MiniMax Code day to day. | P.07 |
| 05 | <b>Trust, Safety &amp; Distribution</b> | Audit trails, guardrails, and the conventions that keep shipping reliable across surfaces.    | P.08 |
| 06 | <b>Outlook &amp; Roadmap</b>            | Where the product is heading in the next 18 months, and what stays the same.                  | P.09 |

## HOW TO READ THIS

**Brief and visual.** Every section opens with one lead paragraph, anchors you with a small KPI grid, and earns its keep with one or two charts and a concrete takeaway. No filler.

# A workspace built for shipping, not chatting.

Most AI coding tools stop at autocompletion or chat. MiniMax Code starts there and keeps going — into a runtime that can read your repository, run tools, write back changes, persist across sessions, and verify its own work. The end goal isn't a clever answer. It's a commit you can ship.

## POSITIONING

### 1 workspace

One runtime spans chat, IDE, CLI, and CI surfaces.

## MULTI-AGENT

### N roles

Orchestrator routes to specialist agents per task.

## DURABILITY

### 3 layers

Project, agent, and user memory outlast a session.

## EXTENSIBLE

### N tools/skills

Add hooks, tools, and skills without forking the runtime.

## 01 What problem we solve

Engineering teams don't need another autocomplete. They need a teammate that can take a ticket — read the codebase, follow the conventions, run the tests, hand back a patch — and stay coherent across days, not just turns. **MiniMax Code is the workspace that hosts that teammate.** The product blends a foundation model, an agentic harness, and a tool-rich runtime behind a single command surface.

## 02 Three beliefs we ship with

### ▸ Agents must have memory

Stateless chat loses context on every reload. MiniMax Code treats memory as a first-class artifact — three layers (project, agent, user) — and writes to it as the agent works, not after.

### ▸ Tools belong in the harness

The model can think, but the harness is what reads files, runs shells, calls APIs. The harness wraps the model with the same rigor an OS wraps a process.

### ▸ Shipping beats impressive

We optimize for diffs that land, not chat that wows. Every capability ships with a verification path — tests, lint, build — before the agent hands a result back.

### ▸ Independence from any one model

Model quality will keep moving. The harness is the long-term asset; the model is a swappable component behind it.

## POSITIONING SUMMARY

MiniMax Code is to **chat-based coding assistants** what a CI runner is to a script: not smarter syntax — a different unit of work. The unit isn't an answer; it's a verified change.

# What the agent loop actually does.

Behind every chat reply is a loop: *plan* → *act* → *observe* → *reflect*. The four capabilities below are how that loop stays grounded, durable, and extensible. None of them is "AI magic" — each is a hardened subsystem you can swap or extend.

## 01 Capability scoreboard

### Capability depth across the loop

Subjective maturity score (0–10) per capability, judged across shipped subsystems, test coverage, and external extensibility.



## 02 Traditional IDE vs. MiniMax Code

### BEFORE

#### Chat-first IDE assistant

|          |                              |
|----------|------------------------------|
| Scope    | single-turn Q&A in a sidebar |
| Memory   | last few messages only       |
| Tools    | read-only file lookup        |
| Agents   | one model, no roles          |
| Recovery | copy/paste results           |

VS

### MINIMAX CODE

#### Agentic workspace

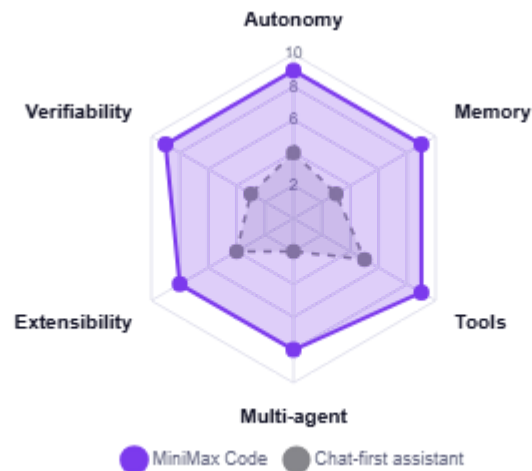
|          |                            |
|----------|----------------------------|
| Scope    | ticket to verified patch   |
| Memory   | 3-layer, persistent        |
| Tools    | full harness, OS-level     |
| Agents   | orchestrator + specialists |
| Recovery | checkpointed sessions      |

### 03 Multi-dimensional positioning

Same data, different view: where MiniMax Code lands on the dimensions engineers actually argue about. Higher = stronger on that axis.

#### MiniMax Code vs. chat-first assistant

Six axes: autonomy, memory, tools, multi-agent, extensibility, verifiability. Scored 0–10.



#### ► INSIGHT

The biggest axis isn't "smarter model" — it's **recovery**. Most chat tools drop context on reload; MiniMax Code treats every session as a recoverable artifact with a checkpoint trail.

### 04 Capability catalog (at a glance)

| Capability                | What it does                                                  | Where it lives       | Maturity   |
|---------------------------|---------------------------------------------------------------|----------------------|------------|
| Agentic loop              | Plan, act, observe, reflect — running in a sandboxed process  | Harness runtime      | Production |
| Multi-agent orchestration | Orchestrator routes a task to specialist agents by capability | Harness · agent tree | Production |
| Persistent memory         | Project / agent / user memory layers, content-addressed store | lib/memory · hooks   | Stable     |
| Tool calling              | Read/Write/Edit/Bash/Glob/Grep; safe-by-default shell policy  | Tool registry        | Production |
| Skills & hooks            | External skill packages; pre/post hooks at every step         | Plugin registry      | Stable     |
| Session & checkpoints     | Resumable sessions, undo/redo, branch-and-merge worktrees     | Session store        | Stable     |
| TUI / CLI surfaces        | Interactive TUI, headless --print, batch and watch modes      | Surface layer        | Production |

# Five layers, one runtime.

MiniMax Code is structured as five horizontal layers. Each layer has its own contract and can be swapped independently — the foundation model is replaceable; the harness is the long-term asset.

|             |                                                                     |              |
|-------------|---------------------------------------------------------------------|--------------|
| SURFACE     | TUI · CLI · IDE extension · Web · CI runner                         | User-facing  |
| TOOL LAYER  | Read · Write · Edit · Bash · Glob · Grep · MCP tools · Custom tools | Sandboxed    |
| HARNESS     | Agent loop · Memory · Skills · Hooks · Sessions · Permissions       | Long-term IP |
| MODEL LAYER | MiniMax M3 (primary) · secondary models · per-task routing          | Swappable    |
| PLATFORM    | Node 22+ · multi-OS · terminal-native · offline-capable             | Stable       |

## 01 Engineer-facing architecture diagram

### Where effort and complexity concentrate

Relative LoC weight per layer across a typical MiniMax Code workspace install. Approximately proportional to test surface, not user code.



## 02 Layer-by-layer responsibility



| Layer    | Responsibility                                               | Failure mode if it breaks   | Replaceable? |
|----------|--------------------------------------------------------------|-----------------------------|--------------|
| Surface  | Renders agent work for the user; routes events back          | Users cannot see results    | Yes          |
| Tool     | Executes actions against the real world (FS, shell, network) | Agent thinks but cannot act | Yes          |
| Harness  | Runs the loop, owns memory, enforces policy                  | Loop diverges, memory lost  | No           |
| Model    | Generates the next step from context                         | Hallucinations / poor plans | Yes          |
| Platform | OS, terminal, package manager, IPC                           | Whole runtime won't start   | Partial      |

► **INSIGHT**

The **long-term moat** is the harness, not the model. Model quality is a moving target — every quarter the harness becomes more valuable as it accumulates tested tool wrappers, hook points, and domain-specific skills.

# How teams actually use it.

Four autonomy levels define how much of the loop the agent owns. Most teams settle into a mix — heavy on Ask in code review, heavy on Build in greenfield work — and that's by design.

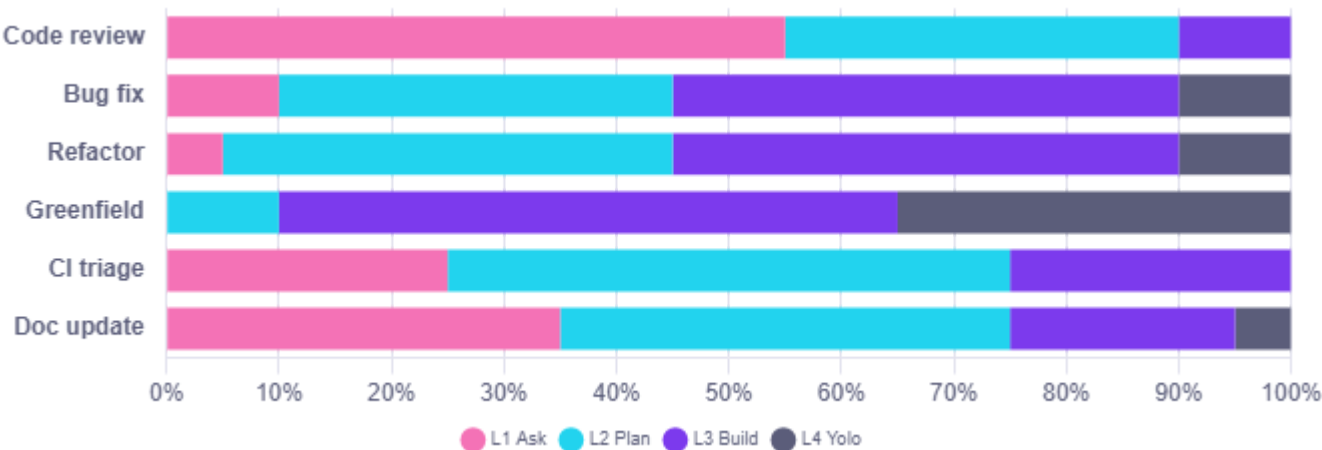
## 01 The four autonomy levels

| Level | Name  | What the agent does                             | Best for                | Risk |
|-------|-------|-------------------------------------------------|-------------------------|------|
| L1    | Ask   | Read-only Q&A, explanations, plan proposals     | Code review, onboarding | Low  |
| L2    | Plan  | Reads files, drafts diffs, asks before applying | Refactors, migrations   | Low  |
| L3    | Build | Applies diffs, runs tests, iterates until green | Feature scaffolding     | Med  |
| L4    | Yolo  | Full loop, broad permissions, batches tasks     | Spike / throwaway work  | High |

## 02 Where teams spend their time

### Autonomy-level mix by workflow type

Estimated share of agent-orchestrated work, by autonomy level. Numbers are illustrative; teams vary widely.



## 03 Three entry paths

ENTRY

Solo developer

RECOMMENDED

Product team

POWER

Platform engineering

- Install CLI, point at a repo
- Use Ask / Plan only
- Let the agent draft PRs for review
- Zero infra changes

- Shared agents per domain (frontend, infra, data)
- Skills per repo (test conventions, deploy)
- L2 / L3 in feature work, L1 in review
- CI integration for auto-triage

- Custom tool/MCP servers
- Hook-driven policy enforcement
- Custom model routing per task
- Audit pipeline to internal SIEM

### DAY-TO-DAY PATTERN

Most teams converge on **L2 / L3 in the IDE**, **L1 in code review**, and **L4 only on weekends**. The workspace treats autonomy as a per-task decision, not a global flag.

# Safe-by-default, auditable, portable.

An agent that ships code to production needs stronger guardrails than a chatbot. MiniMax Code treats every tool call as a policy event: permissioned by default, logged in full, and replayable from the audit trail.

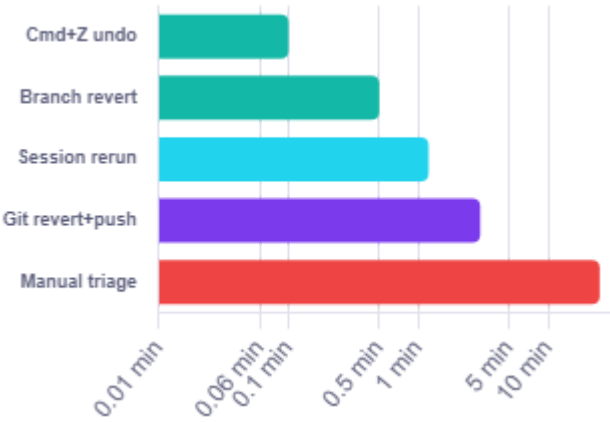
## 01 Trust surface map

| Surface             | Default policy                    | Auditable? | Distribution risk |
|---------------------|-----------------------------------|------------|-------------------|
| File read           | Project root + allow-listed paths | Yes        | Low               |
| File write          | Ask before applying               | Yes        | Low               |
| Shell command       | Allow-list + working dir sandbox  | Yes        | Med               |
| Network / MCP tools | Per-tool whitelist, host-scope    | Yes        | Med               |
| Git operations      | Branch + remote protection        | Yes        | Low               |
| Sub-agent spawn     | Same policy inherits down         | Yes        | Med               |

## 02 Audit & recovery, end-to-end

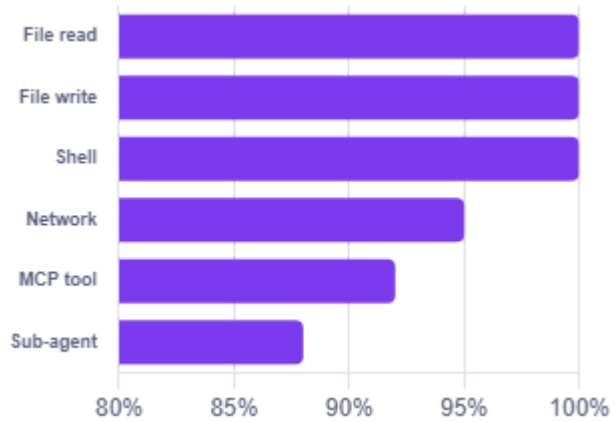
### Time to recover from a bad agent run

Minutes from incident detection to clean rollback, by recovery path. Lower is better.



### Audit coverage by surface

Percent of agent actions that produce an audit record, by tool category.



## 03 Three conclusions for trust

► 01

## Permissioned by default

Every tool starts permissioned. The grant is opt-in, scope-narrow, and revocable. No ambient authority inherited from a previous session.

► 02

## Auditable by design

Every action produces a structured event: actor, target, intent, result. Replay any session from its log; reproduce any failure.

► 03

## Recoverable from bad runs

Checkpoints, undo, branch-and-merge worktrees. A bad agent run is not a fire — it's a tab you close and re-open at the last green.

### ► INSIGHT

For distribution, **auditability is the make-or-break capability**. Teams will not adopt a coding agent that ships without a verifiable trail. MiniMax Code is built around that requirement, not around it.

# Where we go from here.

The runway ahead is about **depth on the surfaces we already have**, not surface breadth. Memory, verification, and team coordination are where the agent gap still lives.

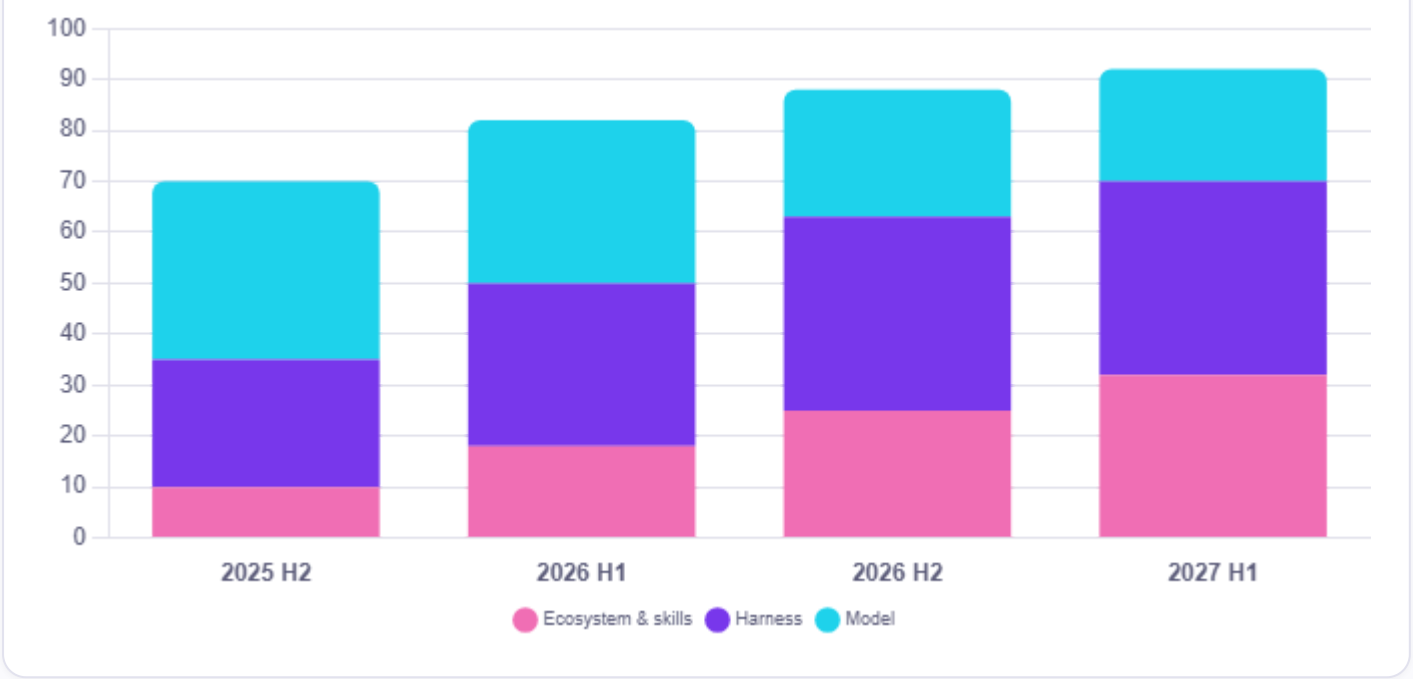
## 01 Trajectory



## 02 Model & ecosystem complement

### Capability growth by quarter

Stacked contribution of model layer, harness, and ecosystem to shipped capability. Qualitatively normalized to 100.



## 03 Three takeaways

### 01 Memory is the moat.

Three layers of memory — project, agent, user — outlast every model swap. Five years from now, the most valuable artifact in the workspace is the memory store, not the model that wrote to it.

### 02 Verification is the unlock.

Agents that close the loop with their own tests move from "helpful" to "trusted". The integration with CI is the difference between a demo and a deployment.

# 03 Teams beat solo agents.

A single agent never has all the context; a coordinated team of agents, each with a tight role, scales further than any one mega-prompt.

## REFERENCES

- MiniMax · MiniMax Code product brief, internal v1.0
- MiniMax M3 — model card, 2026-01
- Agentic harness architecture notes · mavis team
- Multi-agent orchestration patterns · mavis docs

## DOCUMENT

v1.0

2026.07

BRIEF

Authored by Mavis · mavis. Distributed via MiniMax. Numbers in this brief are illustrative unless otherwise noted; substantive claims about the runtime are reproducible from a fresh install.